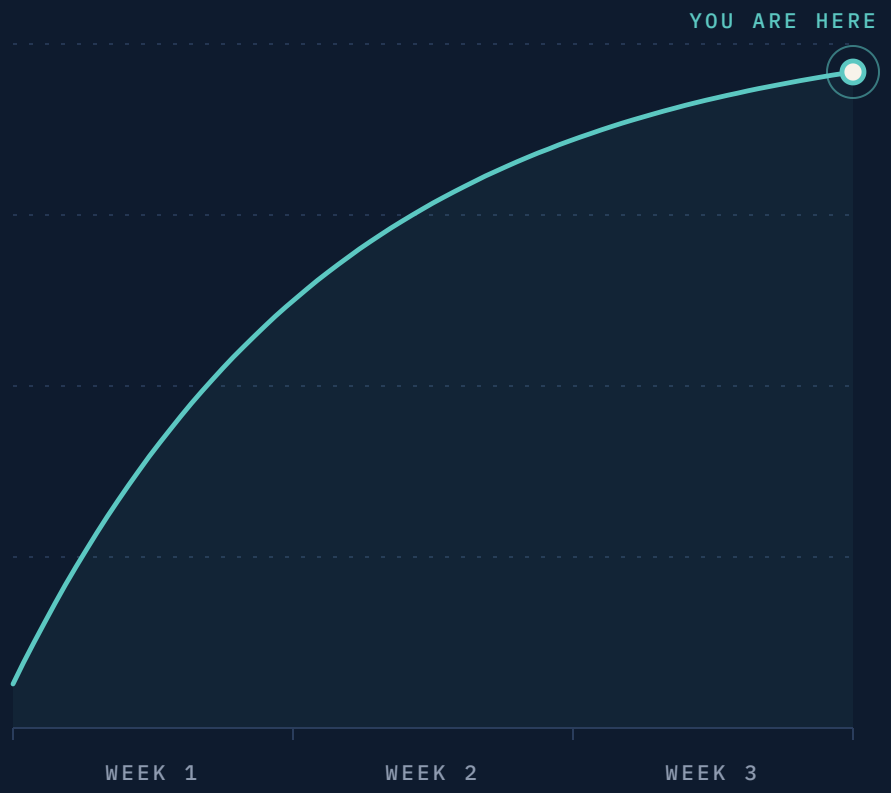
WEEK 3 · DAY 5 · OVERVIEW

Course 2 Capstone



Created by **Jeff Lenamon**

ABOUT 40 MIN / 11 SECTIONS

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Brief

Month end. Operations sent November's extract with a cover note. Before any number leaves the desk, prove the file: every column repriced with your own bondmath, every difference explained, the fixes in one function so next month is one call. Then the risk report as a workbook: DV01, duration, spread and DTS by sector and rating, and the rate and spread what-ifs. Run the same functions on September and show me what changed. One page. Describe it: no views on any bond, issuer, or sector.

ABOUT 40 MIN / 11 SECTIONS

Your Route Through Day 5

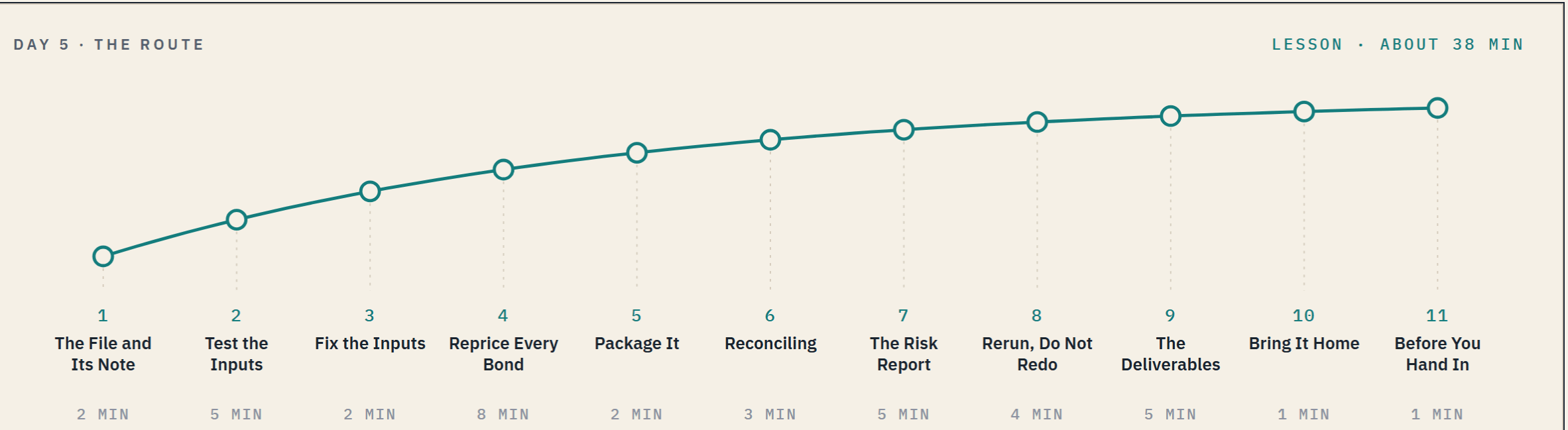


FIGURE 2 Eleven sections, in order, with about how long each takes.

Time. About 38 minutes for the lesson.

1 The File and Its Note

THE COVER NOTE

```
Subject: Month-end holdings extract, as of 2026-11-30
  Number of bonds:      199
  Total face:           493,000,000 dollars
  Total market value:   496,372,835.00 dollars
```

THE FILES

FILE	WHAT IT IS
`course2_capstone_holdings.csv`	November's extract, in the 19 columns Course 1's project pictured
`course2_capstone_cover_note.txt`	the cover note above
`benchmark.csv`	821 bonds as of 2026-09-30

WHAT THE SECTION ASKS

QUESTION	WHAT IT ASKS
1.1 Rows, Columns, and a Working Copy	What shape is the extract (**answer_1_1**)? Copy **`raw`** to **book**: every fix from here goes to **book**.
1.2 The File Less Its Note	Print **`headline()`** as received, then the file less the note in rows, face (**answer_1_2**), and market value. What do the gaps suggest?

Write down the row count: the cover note says 199.

2 Test the Inputs

QUESTION	WHAT IT ASKS
2.1 Rule 1: One Row for Each Bond	How many rows copy an earlier row exactly (**answer_2_1**)? Show each beside its original, with the face and value they carry.
2.2 Rule 2: The Maturity	How many rows mature on or before the as-of date (**answer_2_2**)? Show them with coupon, maturity, and duration: which column is wrong?
2.3 Rule 3: Missing Numbers	How many cells are missing (**answer_2_3**)? Which of each row's other numbers gives the blank back through your module?
2.4 Rule 4: Units	Summarize `coupon` and `yield_pct`. How many coupons (**answer_2_4a**) and yields (**answer_2_4b**) are below 1 percent? Show those rows with price and accrued.
2.5 Rules 5 and 7	How many rows miss rule 5 by more than half a cent (**answer_2_5a**), and how many could be tested? Total the breaks of rule 7 (**answer_2_5b**).

Every rule gets a count: failures are the fix list, passes go in the page.

3 Fix the Inputs

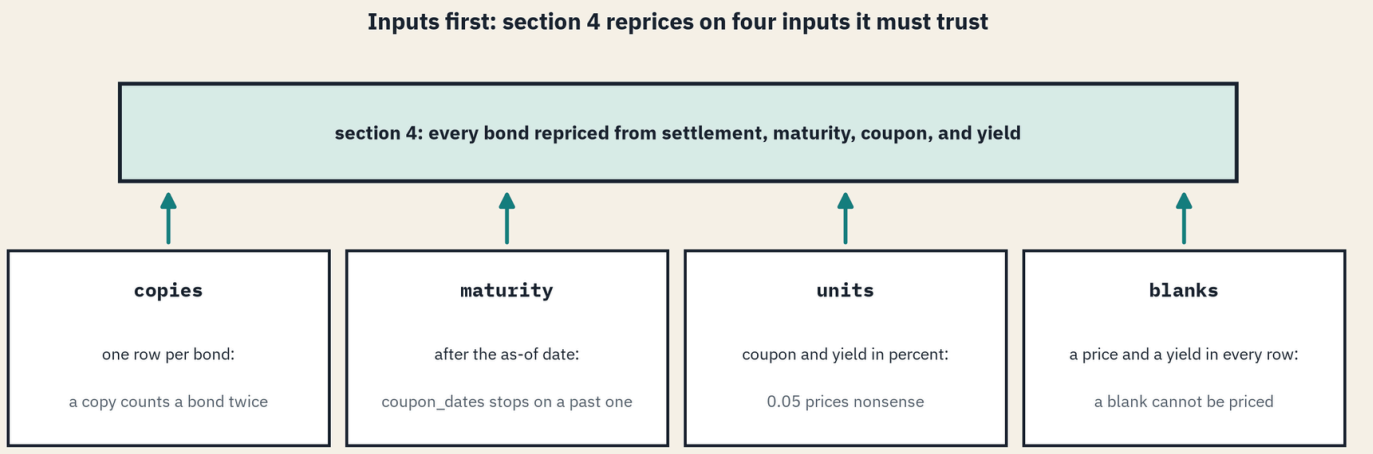


FIGURE 3 Four inputs come first: one row per bond, a maturity after the as-of date, units in percent, and no blanks.

WHAT THE SECTION ASKS

QUESTION	WHAT IT ASKS
3.1 Copies, Maturity, and Units	Make your three fixes. Give the face after them (**answer_3_1a**), the corrected maturity as text year-month-day (**answer_3_1b**), and the mean coupon to 3 places (**answer_3_1c**).
3.2 Fill the Blanks	Fill each blank from the other number of its pair. Give the sum of the yields you filled (**answer_3_2a**) and of the prices (**answer_3_2b**), each to 3 places.

Totals that tie cannot see a wrong price; section 4 can. The first sitting ends here.

4 Reprice Every Bond

QUESTION	WHAT IT ASKS
4.1 Accrued Interest	Add **`accrued_calc`**. How many rows of **`accrued`** are beyond tolerance (**answer_4_1**)? Set each beside coupon x actual days since the last coupon / 360. Decide and fix.
4.2 Price and Yield	Add **`price_calc`**. How many prices are beyond tolerance (**answer_4_2a**)? Compare each with **`price_calc`** plus accrued; decide and fix. Then give the largest gap of **`yield_calc`**, from the price, to **`yield_pct`**, to 4 places (**answer_4_2b**).
4.3 Duration	Add **`duration_calc`**. How many durations are beyond tolerance (**answer_4_3**)? Set each beside **`spread_duration`** and **`macaulay_duration`**. Decide and fix.
4.4 The Two Special Bonds	Print the month-end bond's **`coupon_dates`**, 30/360 days accrued, and accrued, and give its next coupon date as text (**answer_4_4a**). Print the final-period bond's A, E, and DSC, and give its dirty price to 6 places (**answer_4_4b**).

EXCEL SIDE BY SIDE

JOB	PYTHON	EXCEL
Month-end bond, next coupon	`coupon_dates(...)[1]`	`=COUPNCD(DATE(2026,11,30),DATE(2033,8,31),2,0)`
Its clean price	`price(*bond)`	`=PRICE(DATE(2026,11,30),DATE(2033,8,31),5.5/100,5.7/100,100,2,0)`
Final-period bond, clean price	`price(*bond)`	`=PRICE(DATE(2026,11,30),DATE(2027,3,15),4.25/100,4.193/100,100,2,0)`

Reprice Every Bond: Dirty Price to DTS

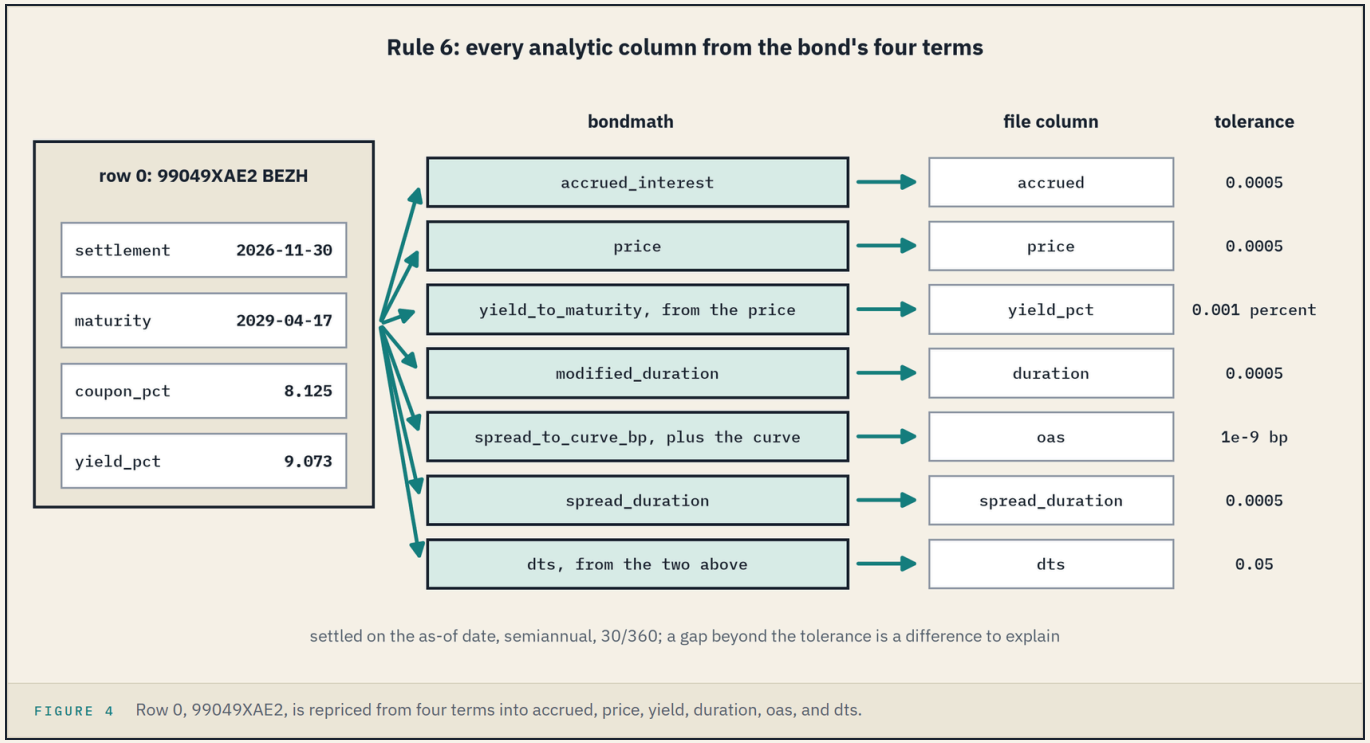


FIGURE 4 Row 0, 99049XAE2, is repriced from four terms into accrued, price, yield, duration, oas, and dts.

WHAT THE SECTION ASKS

QUESTION	WHAT IT ASKS
4.5 Dirty Price, DV01, and Convexity	Add **`dirty_price`**, **`dv01`**, and **`convexity`**, and check each DV01 against notebook 09's bump, leaving out the final-period bond. Give the book's DV01 in dollars, to 2 places (**answer_4_5**).
4.6 Spread to the Curve, Two Ways	Add **`spread_bp`** on notebook 12's formula curve, and **`spread_bp_table`** on **`curve`**. How many **`spread_bp`** are more than 1e-9 bp from **`oas`** (**answer_4_6a**)? Give the largest gap of **`spread_bp_table`** to **`oas`**, to 2 places (**answer_4_6b**).
4.7 Spread Duration and DTS	Add **`spread_duration_calc`**, and **`dts_calc`** from it and **`spread_bp`**. How many **`spread_duration`** (**answer_4_7a**) and **`dts`** (**answer_4_7b**, tolerance 0.05 + 1e-9) are beyond tolerance? Recompute DTS from the file's own **`spread_duration`**: what explains them?

5 Package It

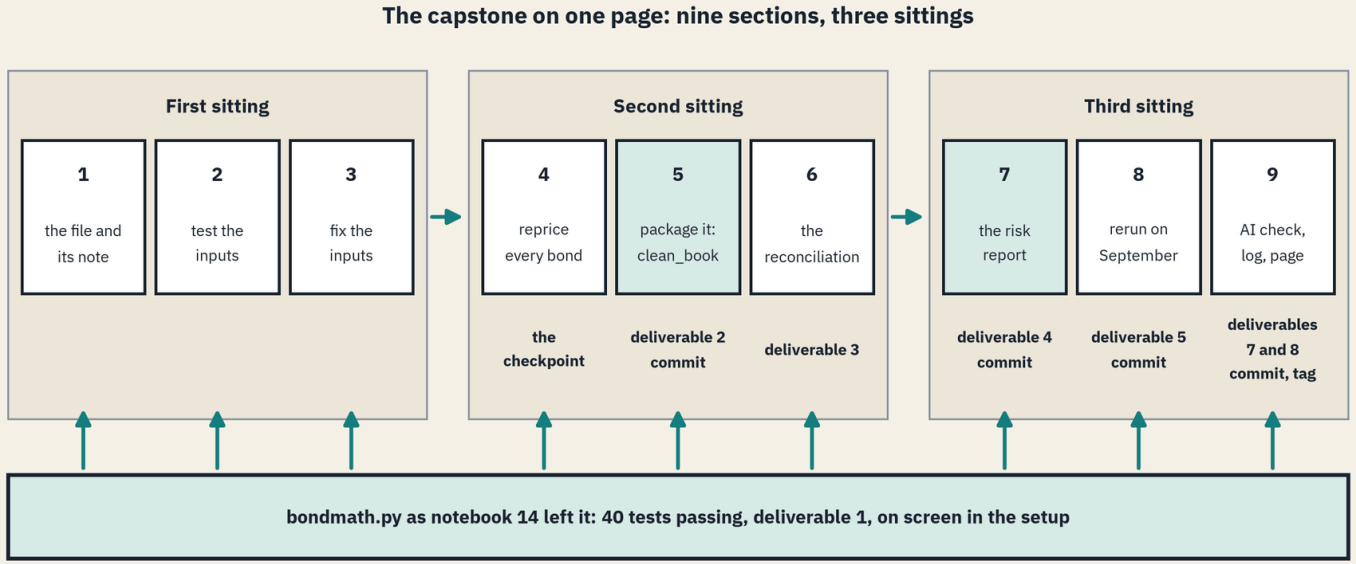


FIGURE 1 Nine marked sections in three sittings, with 40 module tests behind them and four commits and a tag on the way.

WHAT THE SECTION ASKS

QUESTION	WHAT IT ASKS
5.1 `clean_book`, Docstring First	In `capstone.py`, written with `%%writefile` in a cell of its own, write `clean_book(raw, benchmark, curve)`: a docstring listing your seven decisions, then the decisions and section 4's columns, its inputs unchanged. `assert` each rule it promises, with a message. Store its result on `raw` in **cleaned**, `assert` the control totals, and show `cleaned` equals **book**.
5.2 Commit It	Commit `capstone.py` alone, with a subject naming the deliverable, and show the history.

One call rebuilds the book from the file as it arrived.

6 The Reconciliation

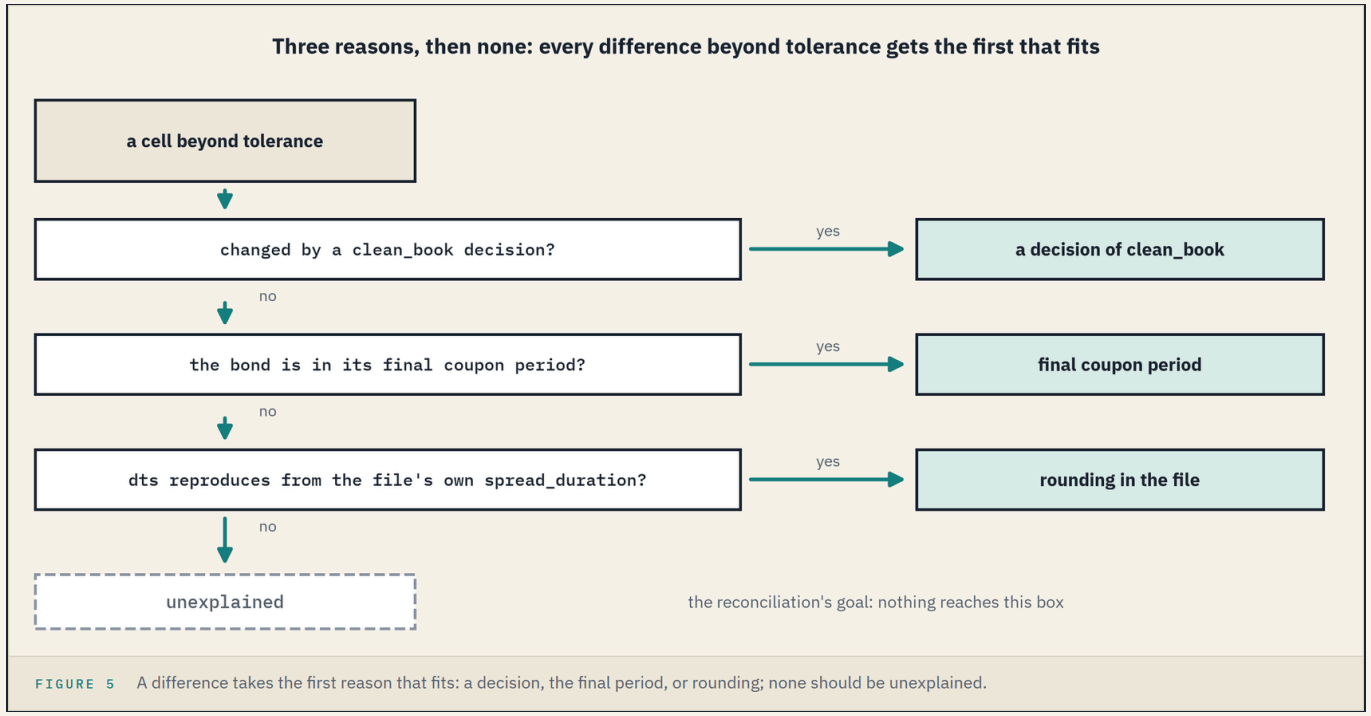


FIGURE 5 A difference takes the first reason that fits: a decision, the final period, or rounding; none should be unexplained.

WHAT THE SECTION ASKS

QUESTION	WHAT IT ASKS
6.1 Delivered Against Recomputed	Line up the extract, without its copies, row for row with **`cleaned`**. Count the cells beyond tolerance (**answer_6_1**): **`coupon`** and **`maturity`** against the cleaned values, and the eight other analytic and value columns against what **`bondmath`** recomputed. A blank is a difference.
6.2 A Reason for Every Difference	Build **reconciliation**, one row per difference: CUSIP, column, delivered, recomputed, and the first reason in the picture's order. How many are unexplained (**answer_6_2a**), and how many bonds did a decision change (**answer_6_2b**)?

Every difference has a name. The second sitting ends here.

7 The Risk Report

QUESTION	WHAT IT ASKS
7.1 `risk_report`	Append `risk_report(book, ratings)` to `capstone.py` with `%%writefile -a`. It returns a dict of three tables:
7.2 By Sector and Rating	Which sector carries the largest share of the book's DTS (**answer_7_2a**)? What percent of the DTS is High Yield, to 2 places (**answer_7_2b**)? Set each beside its share of value.
7.3 The What-Ifs	Give the +100 bp full reprice in dollars (**answer_7_3a**), and the -100 bp estimate less full reprice (**answer_7_3b**), both to 2 places. Why do the two part for a large move? Each is a mechanical what-if, never a forecast.
7.4 The Workbook	Write `course2_risk_report.xlsx`, a sheet per table, no row labels. Read it back and `assert` the positions sheet adds to the book's DV01 and the note's value. Give the sheet names as a list (**answer_7_4a**) and the positions rows (**answer_7_4b**).
7.5 Commit the Code, Not the Workbook	Read `git status --short` and `git diff --stat`, then commit `capstone.py` alone, leaving the workbook untracked (notebook 14).

EXCEL SIDE BY SIDE

JOB	PYTHON	EXCEL
Book DV01 in dollars	`positions['dv01_dollars'].sum()`	`=SUM(R2:R200)`
Book duration	`bondmath.weighted_average(...)`	`=SUMPRODUCT(N2:N200,P2:P200)/SUM(N2:N200)`
Book DTS	`bondmath.weighted_average(...)`	`=SUMPRODUCT(N2:N200,V2:V200)/SUM(N2:N200)`

8 Rerun, Do Not Redo

QUESTION	WHAT IT ASKS
8.1 September Through Both Functions	Run `clean_book`, then `risk_report`, on `september`, unchanged. Do its delivered columns come back untouched? Give September's book DV01 in dollars, to 2 places (**answer_8_1**).
8.2 A Test for Each Promise	Write `test_capstone.py` with `test_clean_book_ties_to_the_cover_note` (November's three totals) and `test_clean_book_leaves_september_alone`. Run pytest below, give the count passed (**answer_8_2**), and commit the test file.
8.3 Two Months Side by Side	Build **comparison**, columns `September`, `November`, and `change`, with a row each for bonds, market value, DV01, duration, spread, and DTS. Give the DTS change, November less September, to 1 place (**answer_8_3**).
The Change in Percent	Which figures moved most from September? This cell is written for you.

> **Two months, one call each, and a test that says so.**

Check an assistant's code, then write the page.

9 The Deliverables

QUESTION	WHAT IT ASKS
9.2 The AI-Use Log	Write one row per question you asked an AI assistant; **`none used`** is valid. Run the cell to write **`ai_use_log.md`**.
9.3 The Page	Write the page for the portfolio manager, 300 to 450 words, in five parts:
Commit, Tag, and Count	Commit the log and the page together, tag the commit **`course2`**, and show **`git log --oneline --decorate`**. How many commits are there (**answer_git**)?

THE FIVE PARTS

- the book in numbers, with units;
- the risk by sector and rating;
- each what-if, full reprice against estimate;
- September against November;
- the data: each problem, its rows, fix, and evidence, and what was clean.

Copy your work into your own module.

Bring It Home

COPY IT HOME

```
Set-Location "$HOME\projects\my_bondmath"
$work = "$env:TEMP\pfi_course2_15_k4w7q2xr"
Copy-Item "$work\capstone.py", "$work\test_capstone.py", "$work\course2_risk_report.xlsx" .
Copy-Item "$work\course2_capstone_holdings.csv", "$work\holdings.csv" .
Copy-Item "$work\benchmark.csv", "$work\course2_treasury_curve.csv" .
python -m pytest -q
git add capstone.py test_capstone.py
git commit -m "Course 2 capstone: clean_book, risk_report, and their tests"
git tag course2
```

The workbook stays untracked, to open in Excel; in Colab, download the files from the file browser.

Before You Hand In

THE RULES

- **Work in a copy:** fixes go to **book**, never to **raw**.
- **State every decision** in a markdown cell: what, how many rows, what evidence.
- **Print `headline()` before and after every fix.**
- **Every analytic comes from `bondmath`,** on the cover note's conventions.
- **Describe:** never recommend, rank, or forecast; a scenario is a what-if.
- **Every Git command names the work folder: `!git -C "{work}" ...`.**
- **Restart and Run All** before you hand in.

HAND IN

- every check **`correct`** and **`raw`** unchanged;
- a decision cell and headlines for every fix;
- both functions with docstrings and **`assert`**s, and **`test_capstone.py`** passing;
- four commits, the tag, the log, and the page.

You Can Now

In notebook 01 your module was 2 functions and 3 tests; today it is 25 and 40, with Excel as the answer key. Since then you have learned to:

- price a bond between coupon dates, clean and dirty, to Excel's last digit;
- measure its duration, DV01, convexity, spread, and DTS;
- add up a book and reprice it under what-ifs;
- keep every function tested and every change in Git;
- prove a file against its note before a number leaves the desk.

NEXT
WEEK 1 · DAY 1

APIs, Keys, and Secrets

What FRED sends back, a key kept outside the code, get_secret tested, and a first live pull tied to the Treasury file.